1. Experiment No. :Experiment No.: 04
2. Student Name : ADESH DEEPAK PATANKAR
3.
4. Class : TYBDCE

Experiment No. 4

Title: Implement matrix multiplication using the MapReduce programming model to perform distributed computation on two input matrices.

Outcome: Students will be able to perform implementation of Matrix Multiplication using MapReduce.

Theory:

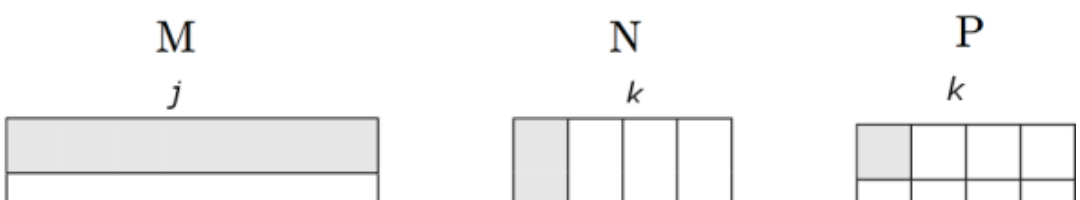
MapReduce is a technique in which a huge program is subdivided into small tasks and run parallelly to make computation faster, save time, and mostly used in distributed systems. It has 2 important parts:

- **Mapper:** It takes raw data input and organizes into key, value pairs. For example, In a dictionary, you search for the word "Data" and its associated meaning is "facts and statistics collected together for reference or analysis". Here the Key is Data and the Value associated with is facts and statistics collected together for reference or analysis.
- **Reducer:** It is responsible for processing data in parallel and produce final output.

Let us consider the matrix multiplication example to visualize MapReduce. Consider the following matrix:

Matrix-vector and matrix-matrix calculations fit nicely into the MapReduce style of computing. In this post I will only examine matrix-matrix calculation as described in [1, ch.2].

Suppose we have a pxq matrix M, whose element in row *i* and column *j* will be denoted m_{ij} and a qxr matrix N whose element in row j and column k is donated by n_{jk} then the product P = MN will be pxr matrix P whose element in row i and column k will be donated by p_{ik} , where $P(i,k) = m_{ij} * n_{jk}$.



Matrix Data Model for MapReduce

We represent matrix M as a relation $M(I, J, V)$, with tuples (i, j, m_{ij}), and matrix N as a relation $N(J, K, W)$, with tuples (j, k, n_{jk}). Most matrices are sparse so large amount of cells have value zero. When we represent matrices in this form, we do not need to keep entries for the cells that have values of zero to save large amount of disk space. As input data files, we store matrix M and N on HDFS in following format:

M, i, j, m_{ij}

```
M,0,0,10.0

M,0,2,9.0

M,0,3,9.0

M,1,0,1.0

M,1,1,3.0

M,1,2,18.0

M,1,3,25.2

....
```

N, j, k, n_{jk}

```
N,0,0,1.0

N,0,2,3.0

N,0,4,2.0

N,1,0,2.0

N,3,2,-1.0

N,3,6,4.0

N,4,6,5.0

N,4,0,-1.0

....
```

MapReduce

We will write Map and Reduce functions to process input files. Map function will produce $key, value$ pairs from the input data as it is described in Algorithm 1. Reduce function uses the output of the Map function and performs the calculations and produces $key, value$ pairs as described in Algorithm 2. All outputs are written to HDFS.

Algorithm 1: The Map Function

1 **for** *each element* m_{ij} *of* M **do**
2 produce $(key, value)$ pairs as $((i,k),(M,j,m_{ij}))$, for $k = 1, 2, 3, ..$ up to the number of columns of N
3 **for** *each element* n_{jk} *of* N **do**
4 produce $(key, value)$ pairs as $((i,k),(N,j,n_{jk}))$, for $i = 1, 2, 3, ...$ up to the number of rows of M
5 **return** *Set of (key, value) pairs that each key,* (i,k)*, has a list with values* (M,j,m_{ij}) *and* (N,j,n_{jk}) *for all possible values of* j

Algorithm 2: The Reduce Function

1 **for** *each key (i,k)* **do**
2 sort values begin with M by j in $list_M$
3 sort values begin with N by j in $list_N$
4 multiply m_{ij} and n_{jk} for j_{th} value of each list
5 sum up $m_{ij} * n_{jk}$
6 **return** $(i,k), \sum_{j=1} m_{ij} * n_{jk}$

The value in row *i* and column *k* of product matrix *P* will be:

$P_{(i,k)} = \sum_{j=1} m_{ij} * n_{jk}$.

Let me examine the algorithms on an example to explain the algorithms better. Suppose we have two matrices, *M*, 2×3 matrix, and *N*, 3×2 matrix as follows:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix}$$

The product *P* of *MN* will be as follows:

$$\begin{bmatrix} 1a+2c+3e & 1b+2d+3f \\ 4a+5c+6e & 4b+5d+6f \end{bmatrix}$$

The Map Task

For matrix *M,* map task (Algorithm 1) will produce $key, value$ pairs as follows:

$(i,k),(M,j,m_{ij})$

$m_{11}=1$
$(1,1),(M,1,1)k=1$
$(1,2),(M,1,1)k=2$

$m_{12}=2$
$(1,1),(M,2,2)k=1$
$(1,2),(M,2,2)k=2$
.....

$m_{23}=6$
$(2,1),(M,3,6)k=1$
$(2,2),(M,3,6)k=2$

For matrix *N,* map task (Algorithm 2) will produce $key, value$ pairs as follows:

$(i,k),(N,j,n_{jk})$

$n_{11}=a$
$(1,1),(N,1,a)i=1$
$(2,1),(N,1,a)i=2$

$n_{21}=c$
$(1,1),(N,2,c)i=1$
$(2,1),(N,2,c)i=2$

$n_{31}=e$
$(1,1),(N,3,e)i=1$
$(2,1),(N,3,e)i=2$
.....

$n_{32}=f$
$(1,2),(N,3,f)i=1$
$(2,2),(N,3,f)i=2$

After combine operation the map task will return $key, value$ pairs as follows:

$$((i,k),[(M,j,m_{ij}),(M,j,m_{ij}),...,(N,j,n_{jk}),(N,j,n_{jk}),....])$$
$$(1,1),[(M,1,1),(M,2,2),(M,3,3),(N,1,a,(N,2,c),(N,3,e)]$$
$$(1,2),[(M,1,1),(M,2,2),(M,3,3),(N,1,b,(N,2,d),(N,3,f)]$$
$$(2,1),[(M,1,4),(M,2,5),(M,3,6),(N,1,a,(N,2,c),(N,3,e)]$$
$$(2,2),[(M,1,4),(M,2,5),(M,3,6),(N,1,b,(N,2,d),(N,3,f)]$$

Note that the entries for the same key are grouped in the same list, which is performed by the framework. This
output will be stored in HDFS and feed the reduce task as input.

The Reduce Task

Reduce task takes the $key,value$ pairs as the input and process one key at a time. For each key it divides the
values in two separate lists for *M* and *N*. For key $(1,1)$, the value is
the list of $[(M,1,1),(M,2,2),(M,3,3),(N,1,a,(N,2,c),(N,3,e)]$.

Reduce task sorts values begin with *M* in one list and values begin with *N* in another list as follows:

$$list_M = [(M,1,1),(M,2,2),(M,3,3)]$$
$$list_N = [(N,1,a),(N,2,c),(N,3,e)]$$,

then sums up the multiplication of m_{ij} and n_{jk} for each j as follows:

$$P(1,1) = 1a + 2c + 3e$$

The same computation applied to all input entries of reduce task.

$$P_{(i,k)} = \sum_{j=1} m_{ij} * n_{jk}$$

for all *i* and *k* is then calculated as follows:

$$P(1,1) = 1a + 2c + 3e$$
$$P(1,2) = 1b + 2d + 3f$$
$$P(2,1) = 4a + 5c + 6e$$
$$P(2,2) = 4b + 5d + 6f$$

The product matrix *P* of *MN* is then generated as:

$$\begin{bmatrix} 1a+2c+3e & 1b+2d+3f \\ \\ 4a+5c+6e & 4b+5d+6f \end{bmatrix}$$

Experiments

Data

I have setup a single node Hadoop installation with HDFS and run the matrix calculation experiment on this installation. We used 1000 x 100 matrix *M* and 100 x 1000 matrix *N* with sparsity level of 0.3. This means each matrix has about 30K entries. The matrix les *M* and *N* stored in input directory on HDFS and the output of the computation is stored in output directory on HDFS.

Source Code

I developed mapper and reducer classes as Map.java and Reduce.java as well as the main application class called MatrixMultiply.java. As it seen in MatrixMultiply.java code below, in main method the configuration parameters are being set as well as the input/output directories of MapReduce job [2].

MatrixMultiply.java

```
package com.matrix;

import  org.apache.hadoop.conf.*;

import

org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.*;

import org.apache.hadoop.mapreduce.*;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;


public class MatrixMultiply {

   public static void main(String[] args) throws Exception {

         if (args.length != 2) {

        System.err.println("Usage: MatrixMultiply <in_dir> <out_dir>");

        System.exit(2);

      }

         Configuration conf = new Configuration();

      // M is an m-by-n matrix; N is an n-by-p matrix.

      conf.set("m", "1000");

      conf.set("n", "100");
```

```
        conf.set("p", "1000");
        @SuppressWarnings("deprecation")
                        Job job = new Job(conf, "MatrixMultiply");
        job.setJarByClass(MatrixMultiply.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
    }
}
```

Mapper class extends org.apache.hadoop.mapreduce.Mapper class and implements the map task described in Algorithm 1 and creates the $key, value$ pairs from the input files as it shown in the code below:

Map.java

```
package com.matrix;

import org.apache.hadoop.conf.*;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
```

```
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class Map
        extends org.apache.hadoop.mapreduce.Mapper<LongWritable, Text, Text, Text>
        {
                @Override
                public void map(LongWritable key, Text value, Context context)
                                throws IOException, InterruptedException {
                        Configuration conf = context.getConfiguration();
                        int m = Integer.parseInt(conf.get("m"));
                        int p = Integer.parseInt(conf.get("p"));
                        String line = value.toString();
                        // (M, i, j, Mij);
                        String[] indicesAndValue =
                        line.split(","); Text outputKey = new
                        Text();
                        Text outputValue = new Text();
                        if (indicesAndValue[0].equals("M")) {
                                for (int k = 0; k < p; k++) {
                                        outputKey.set(indicesAndValue[1] + "," + k);
                                        // outputKey.set(i,k);
                                        outputValue.set(indicesAndValue[0] + "," +
indicesAndValue[2]
                                                        + "," + indicesAndValue[3]);
                                        // outputValue.set(M,j,Mij);
                                        context.write(outputKey, outputValue);
                                }
                        } else {
                                // (N, j, k, Njk);
```

```
                                        for (int i = 0; i < m; i++) {
                                                outputKey.set(i + "," + indicesAndValue[2]);
                                                outputValue.set("N," + indicesAndValue[1] + ","
                                                                + indicesAndValue[3]);
                                                context.write(outputKey, outputValue);
                                        }
                                }
                        }
}
```

Reducer class, extends org.apache.hadoop.mapreduce.Reducer class and implements the reduce task described in Algorithm 2 and creates the $key, value$ pairs for the product matrix then writes its output on HDFS as it shown in the code below:

Reduce.java

```
package com.matrix;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import
java.io.IOException;
import
java.util.HashMap;

public class Reduce
extends org.apache.hadoop.mapreduce.Reducer<Text, Text, Text, Text> {
        @Override
        public void reduce(Text key, Iterable<Text> values, Context context)
                        throws IOException, InterruptedException {
```

```
        String[] value;
        //key=(i,k),
        //Values = [(M/N,j,V/W),..]
        HashMap<Integer, Float> hashA = new HashMap<Integer, Float>();
        HashMap<Integer, Float> hashB = new HashMap<Integer, Float>();
        for (Text val : values) {
                value = val.toString().split(",");
                if (value[0].equals("M")) {
                        hashA.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));
                } else {
                        hashB.put(Integer.parseInt(value[1]), Float.parseFloat(value[2]));
                }
        }
        int n = Integer.parseInt(context.getConfiguration().get("n"));
        float result = 0.0f;
        float
        m_ij;
        float n_jk;
        for (int j = 0; j < n; j++) {
                m_ij = hashA.containsKey(j) ? hashA.get(j) : 0.0f;
                n_jk = hashB.containsKey(j) ? hashB.get(j) : 0.0f;
                result += m_ij * n_jk;
        }
        if (result != 0.0f) {
                context.write(null,
                                new Text(key.toString() + "," + Float.toString(result)));
        }
}
```

}

Commands:-

start-dfs.sh

start-yarn.sh

jps

hdfs dfs -ls /

hdfs dfs -mkdir /input

hdfs dfs -copyFromLocal /home/hadoop/Downloads/MatrixMultiply-main/input/M /input

hdfs dfs -copyFromLocal /home/hadoop/Downloads/MatrixMultiply-main/input/N /input

hadoop jar /home/hadoop/Matrix.jar com.matrix.MatrixMultiply /input /ex4

hadoop fs -cat /ex4/part-r-00000

Handwritten Questions:

1. Explain the concept of matrix multiplication with an example.
2. Why is MapReduce suitable for matrix multiplication?
3. Describe how Mapper and Reducer divide tasks in matrix multiplication.
4. Write the advantages of using MapReduce for large-scale computations.

Dhruv. A. Sakhare
BDTL

Roll no :- 13
BDCE

Experiment-4

Q1 Explain the concept of matrix multiplication with an example.

⇒ Matrix multiplication is the process of multiplying two matrices by taking the dot product of each row of the first matrix with each column of the second matrix. The number of columns in the first matrix must equal the number of rows in the second matrix.

Example :- $\begin{bmatrix} 1, & 2 \\ 3, & 4 \end{bmatrix} \times \begin{bmatrix} 5, & 6 \\ 7, & 8 \end{bmatrix}$

$$\begin{bmatrix} (1\times5+2\times7) & (1\times6+2\times8) \\ (3\times5+4\times8) & (3\times6+4\times8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

Q2. Why mapreduce suitable for matrix multiplication?

⇒ MapReduce is suitable because matrix multiplication involves a large number of independent calculation. These calculation can be distributed across multiple machines.

- Mapper generates intermidiate key-value pairs for multiplication.
- The framework automatically shuffles and group

related values.

- Reducer calculates the final sum for each matrix element.

Supports parallel processing, making it suitable for very large matrices. It provides fault tolerance if a machine fails.

Q3: Describe how Mapper and Reducer divide task in matrix multiplication.

⇒ Mapper :- The mapper reads elements of matrices A and B and generates intermediate key-value pairs based on the position of the resulting matrix elements. It sends the required values to same reducer.

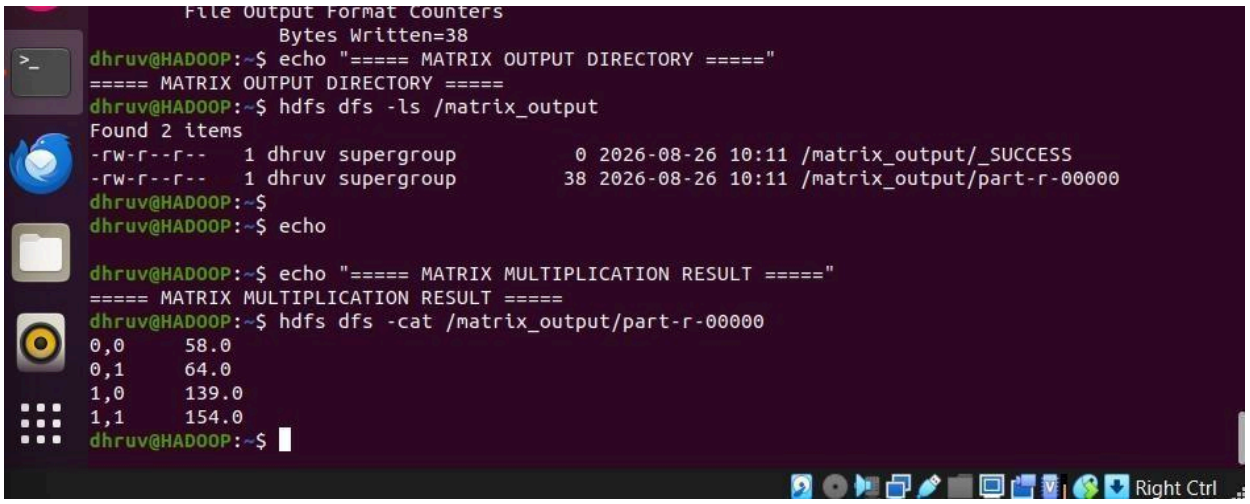
Reducer :- The reducer receives all values belonging to a particular output position multiplies corresponding elements and adds their product. The final sum becomes one elements of the result matrix.

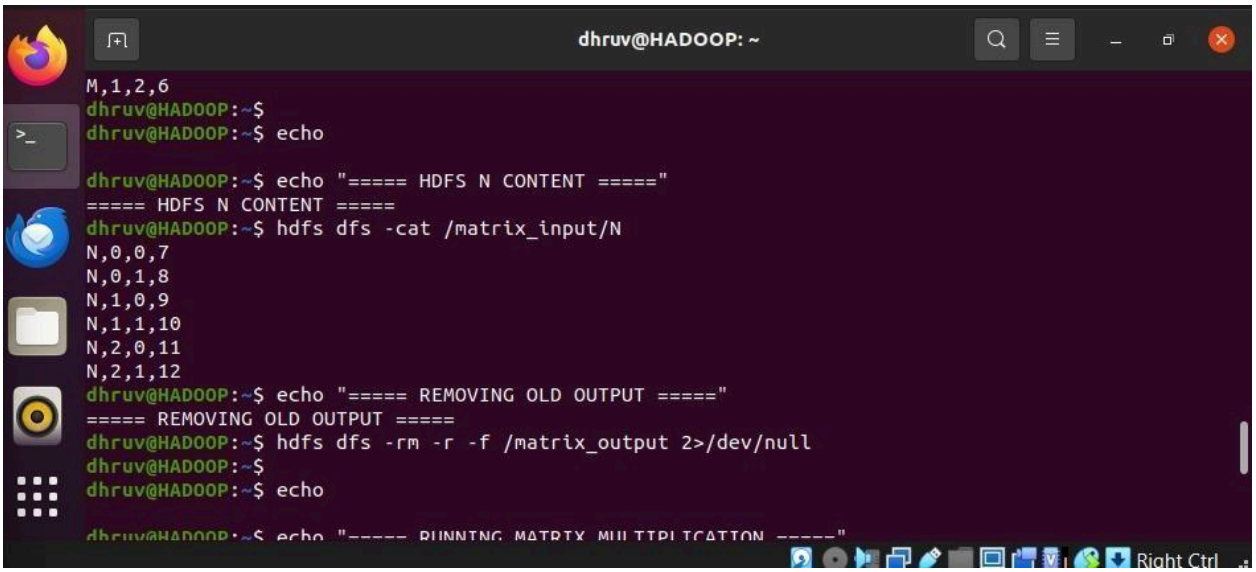
For example :- to calculate C, the reducer computes $C = AB + AB - -$

Thus, the mapper distributes the multiplication work, while the reducer performs the summation and processed seperately.

Acceptance Criteria:

1. Student should load two matrices into HDFS.
2. Student should implement Mapper and Reducer logic for multiplication.
3. Program should produce correct matrix multiplication results in HDFS.
4. Student should display the final output matrix using HDFS commands.





Conclusion : Thus, I have performed implementation of Matrix Multiplication using MapReduce.